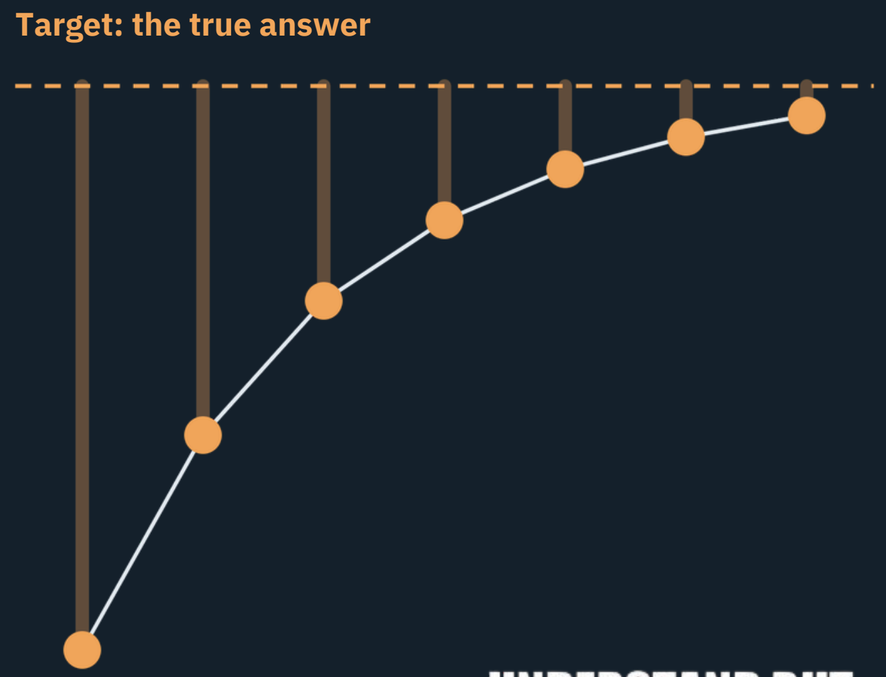
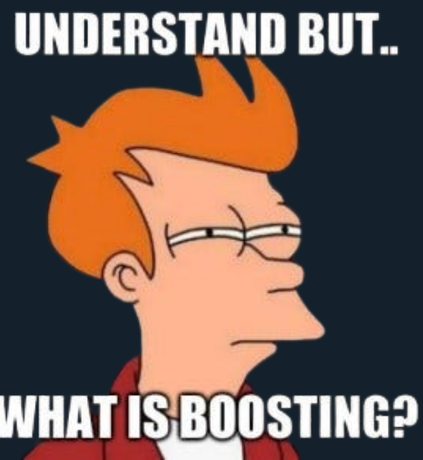
GRADIENT BOOSTING · MACHINE LEARNING LECTURE

XGBoost: Learning From Mistakes

From boosting intuition to real-world Campus Recruitment predictions

Each new tree removes part of the error that’s left





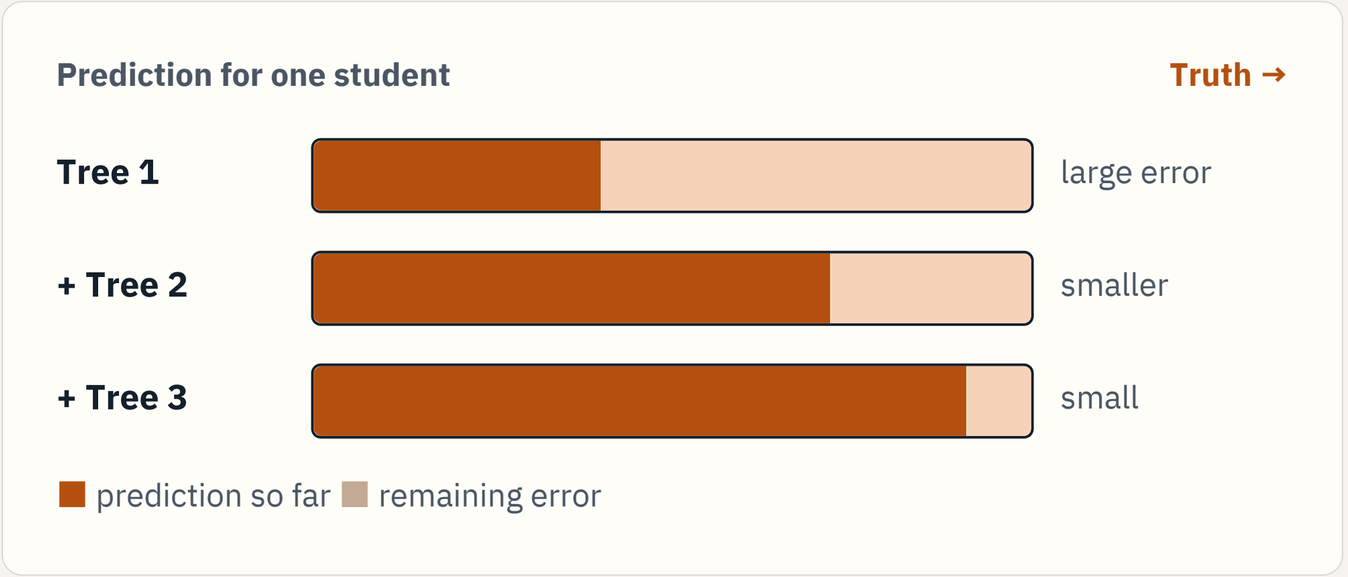
01 · THE IDEA

Boosting: many weak learners, one strong model

1 **Weak learners**
Small, shallow trees — each only a bit better than guessing.

2 **Built in sequence**
Trees are added one after another, never all at once.

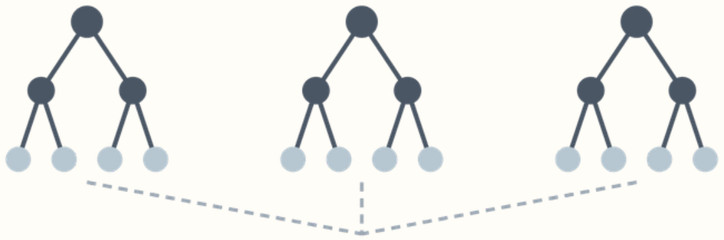
3 **Fix what's still wrong**
Each new tree targets the errors of the current ensemble.



Use it for: Structured, tabular data · Nonlinear patterns · Complex feature interactions

Parallel voters vs. sequential correctors

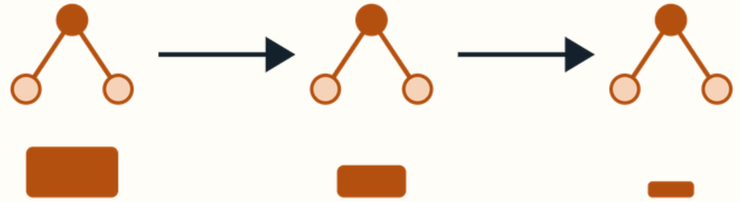
Random Forest (bagging)



Vote / average

- Trees train independently, usually in parallel
- Predictions combined by voting or averaging
- Mainly reduces **variance**

Boosting (sequential)



Tree 1 | Tree 2 fixes 1 | Tree 3 fixes 1 + 2

- Trees are built one after another
- Each new tree corrects the remaining errors
- Mainly reduces **bias**; regularization curbs overfitting

Random Forest asks many trees the same question at once. Boosting asks one tree at a time to fix what the others got wrong.

01 · THE GOLFER ANALOGY

Every shot fixes the distance that’s left



First shot
the initial prediction

Distance left
the prediction error

Next shot
a new tree’s correction

Swing strength
learning rate: how much of it we apply

Many shots
boosting iterations

Three students, one tree, one small step

Student	Score	Actual y	Start p	g = p − y	New p
A	85	1 · Placed	0.69	−0.31	0.695 ↑
B	52	0 · Not placed	0.69	+0.69	0.669 ↓
C	56	0 · Not placed	0.69	+0.69	0.669 ↓

New tree splits on: score < 70 ?

Yes → B, C
G = 1.38 · H = 0.428
w = −0.967 → push down

No → A
G = -0.31 · H = 0.214
w = +0.255 → push up

```
① start in log-odds
z₀ = ln(0.69 / 0.31) = 0.800
② gradient & Hessian
g: A = -0.31 · B, C = +0.69
h = 0.69 × 0.31 = 0.214 (all)
③ split: score < 70 (left)
④ leaf weight w = -G/(H+λ), λ=1
L: -1.38 / 1.428 = -0.967
R: +0.31 / 1.214 = +0.255
⑤ z₁ = z₀ + η·w (η=0.1), p = σ(z₁)
A: 0.800 + 0.026 = 0.826 → 0.695
BC: 0.800 - 0.097 = 0.703 → 0.669
```

Illustrative toy example (logistic loss). Leaf values live in **log-odds**: add η·w to z, then convert with σ(z) = 1 / (1 + e^(−z)). Never add a leaf value directly to a probability.

02 · UNDERSTANDING OUR DATASET

215 MBA students, one question: placed or not?

215
students (rows)

15 → 12
original columns → predictive features

0
missing values after dropping salary

0
duplicate records

Target: status · binary classification



Placed (1) · 148 students — **Not placed (0) · 67**

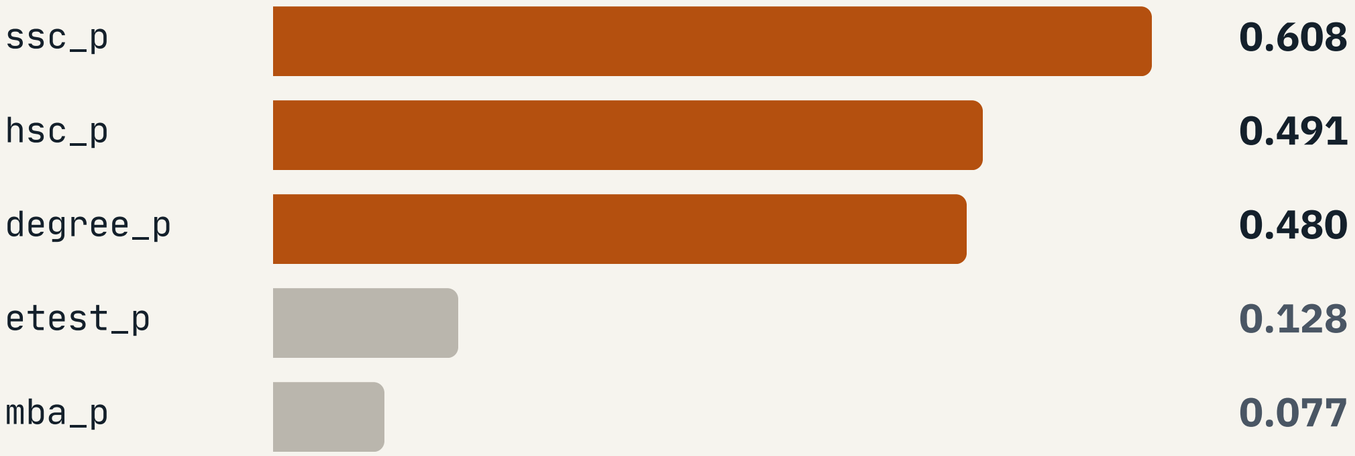
Imbalanced: a model that says “Placed” for everyone is already 69% accurate — so accuracy alone can mislead.

Dropped: salary (leakage)
Recorded only for placed students (148 values = 148 placed). It reveals the answer.

Dropped: sl_no
A row ID. It says nothing about the student — only noise.

Earlier grades matter more than MBA scores

Correlation with placement (Pearson r, sorted)



10th grade · 12th grade · undergrad degree · employability test · MBA

Median 10th-grade score (ssc_p)



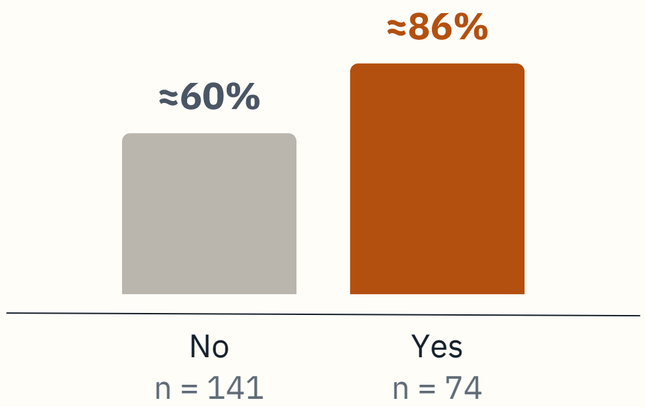
Read from the notebook's boxplots. For etest_p and mba_p, the two groups' boxes overlap heavily.

Insight: earlier academic performance shows a stronger relationship with placement than MBA % or the employability test — in this dataset.

Correlation shows association, not causation — and Pearson r only sees straight-line links, not thresholds.

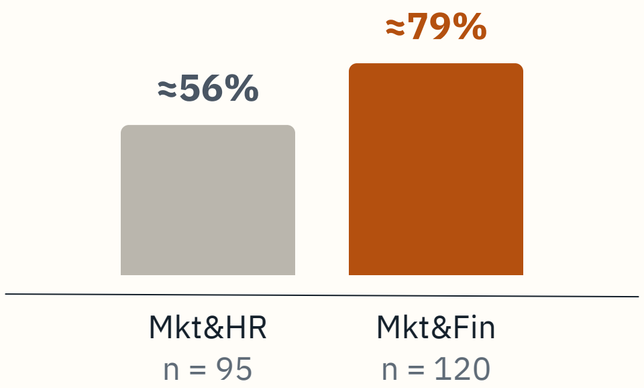
Experience and specialisation shift the odds

Work experience



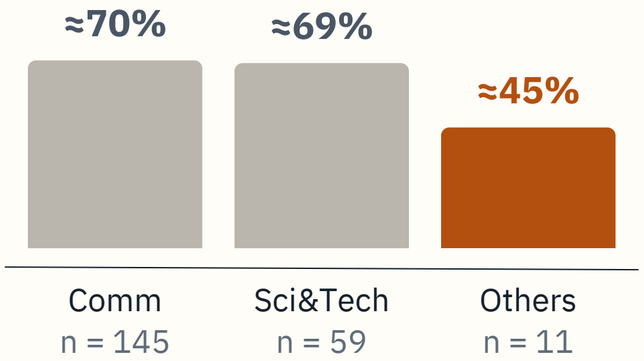
Biggest gap: experienced students are placed far more often.

MBA specialisation



Mkt&Fin is placed ≈23 points more often than Mkt&HR.

Undergraduate degree



“Others” is lowest — but it’s only 11 students.

What EDA taught us: the signal sits in score thresholds and in combinations of scores with categories — the step-shaped, interacting patterns trees learn natively.

Placement rate per group, from the notebook’s bar charts. Group-level associations — not causes, and not verdicts on individuals.

Four formulas decide every tree

① Objective = loss + complexity penalty

$$Obj = \Sigma_i \, \ell(y_i, \hat{y}_i) + \Sigma_k \, \Omega(f_k)$$
$$\Omega(f) = \gamma \cdot T + \tfrac{1}{2} \cdot \lambda \cdot \Sigma_j \, w_j^2$$

ℓ = log-loss (how wrong) · T = number of leaves · w = leaf values. Bigger, bolder trees cost more.

② Gradient and Hessian

$$g = p - y \text{ direction}$$
$$h = p \cdot (1 - p) \text{ curvature}$$

g says which way to correct and how strongly; h says how sharply the loss bends — a Newton-style step.

③ Optimal leaf weight

$$w^* = -G / (H + \lambda)$$

G, H = sums of g and h over the students in a leaf. λ in the denominator shrinks every correction toward 0.

④ Split gain

$$Gain = \tfrac{1}{2} \cdot [\, G_L^2/(H_L+\lambda) + G_R^2/(H_R+\lambda) - (G_L+G_R)^2/(H_L+H_R+\lambda) \,] - \gamma$$

L, R = left and right child. Split only if Gain > 0: the improvement must beat γ, the cost of an extra leaf. Our 3 students: 0.358.

Fast to train, built to resist overfitting

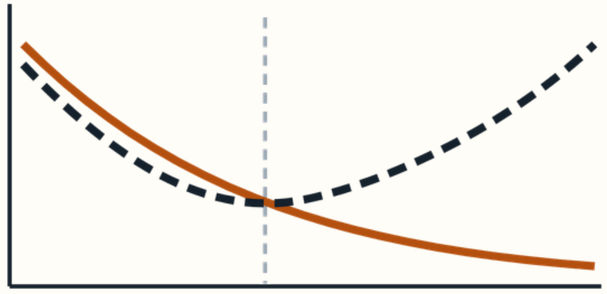
Speed

- Histogram-based split search (tree_method="hist")
- Parallel computation inside each tree
- Optimized, cache-aware CPU code
- Optional GPU acceleration

Overfitting controls

- L1 / L2 penalties (α, λ)
- Maximum tree depth
- Learning rate (shrinkage)
- Row & feature subsampling
- Minimum child weight
- Split penalty γ
- Early stopping

Error vs. model complexity



Complexity → ¦ sweet spot

— training --- validation

These controls reduce the risk of overfitting — none of them guarantees it. A held-out validation set is still the judge.

A leakage-safe pipeline in six steps

1
Dataset
`215 × 15`

→

2
Leakage removal
`drop salary, sl_no`

→

3
Stratified split
`stratify=y`

→

4
Preprocess
`Scaler + OneHot`

→

5
Train XGBoost
`XGBClassifier`

→

6
Validate
`early stop, threshold`

Train · 137 students
learns the trees

Valid · 35
practice test

Test · 43
final exam, once

Stratify
Every split keeps the 69 / 31 class balance.

One-hot
Turns categories into 0/1 columns trees can split on.

Scaling
Not needed for trees — splits use order only. Harmless here.

Fit on train
Scaler and encoder learn from training rows only: no leakage.

Validation
Drives early stopping and picks the threshold.

Small steps, shallow trees, and an early stop

Hyperparameter	Value	In plain words
n_estimators	2000	Upper limit; early stopping picks the real number
learning_rate	0.05	Small, careful steps (η)
max_depth	3	Shallow trees: simple rules only
min_child_weight	3	No leaf built on too little evidence
subsample	0.8	Each tree sees 80% of students
colsample_bytree	0.8	Each tree sees 80% of features
reg_lambda	1.0	L2 penalty λ shrinks leaf values
tree_method	hist	Fast histogram split search
eval_metric	auc	Score watched on validation
early_stopping_rounds	100	Stop after 100 rounds with no gain

TRAINING OBSERVATION

3 **best iteration**

```
[0] valid AUC 0.926
[103] valid AUC 0.894
Best iteration: 3
```

Validation AUC peaked almost at once (index 3, so 4 trees). With only 35 validation students the AUC is noisy — an early peak, not proof of a perfect model.

05 · THE DECISION THRESHOLD

Why we moved the cut-off from 0.5 to 0.7

The model outputs a **probability**. The threshold turns it into a **label**: predict Placed when p ≥ threshold.

All 43 test probabilities: ≈0.59 – 0.74



VALIDATION set · 35 students

Threshold	Macro F1	Balanced acc.
0.50	0.407	0.500
0.60	0.497	0.545
0.65	0.790	0.777
0.70 ★ best	0.873	0.892
0.75 and up	0.239	0.500

Why 0.5 failed

With only 4 trees, every probability sat above 0.5 — so every student was labelled Placed.

Chosen on validation

Highest macro F1 among tested thresholds. The test set stays untouched until the next slide.

05 · TEST SET · 43 STUDENTS

Threshold 0.5 vs 0.7: what actually changed

Threshold 0.5

	Pred. not placed	Pred. placed
Not placed	0	13
Placed	0	30

Everyone predicted Placed: **0 of 13** non-placed students found.

Metric	0.5	0.7
Accuracy	0.70	0.65
Macro F1	0.41	0.64
Recall · Not placed	0.00	0.77
Recall · Placed	1.00	0.60
ROC-AUC	0.827	0.827

Rows = actual, columns = predicted, ordered [Not placed, Placed].

Threshold 0.7

	Pred. not placed	Pred. placed
Not placed	10	3
Placed	12	18

10 of 13 non-placed found — at the cost of 12 missed placed students.

Higher accuracy does not always mean a better model for the purpose.

ROC-AUC unchanged: the threshold changes labels, not the ranking.

05 · FINAL RESULTS · TEST SET · THRESHOLD 0.7

The final model catches 10 of 13 at-risk students

0.827
ROC-AUC · ranking quality

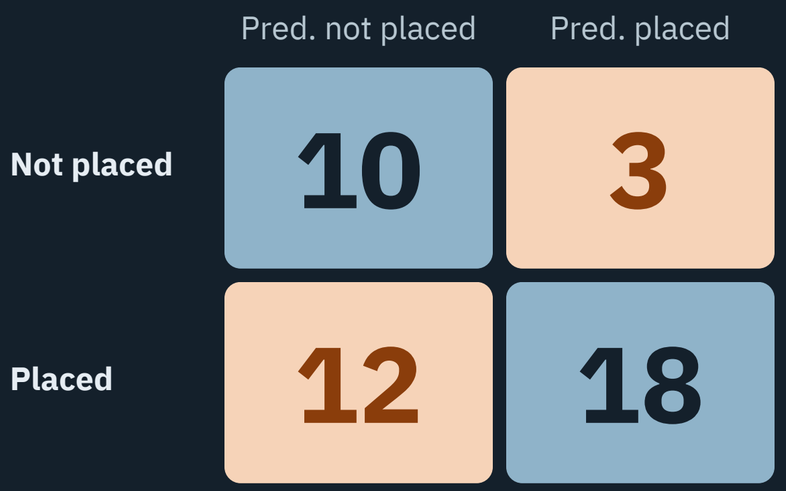
0.651
Accuracy · 28 of 43

0.64
Macro F1 · both classes

0.77
Recall · Not placed (10 / 13)

0.86
Precision · Placed (18 / 21)

15
of 43 predictions wrong



Blue = correct · peach = error. Rows actual, columns predicted.

⚠ **Limitation:** only 43 test students — a different split could move these numbers noticeably. A teaching model, not a deployment-ready tool.

06 · TRADE-OFFS

Powerful on tables, but not a free lunch

Strengths

- Excellent candidate for structured tabular data
- Learns nonlinear relationships
- Captures feature interactions
- Built-in regularization
- Efficient training implementation
- Handles missing values natively
- Early stopping support

Weaknesses

- Sensitive to hyperparameter choices
- Can overfit very small datasets
- Harder to interpret than a single tree
- Boosting rounds can’t run in parallel
- Not suited to raw images or audio
- Can’t extrapolate trends beyond the training range

Reach for XGBoost when…

data is a table of mixed features, patterns are nonlinear, and you can validate carefully.

Try a simpler model first when…

data is tiny (like our 215 rows), or every decision must be fully explainable — e.g., logistic regression.

KEY TAKEAWAYS

Four things to remember

1

Boosting learns **sequentially**, each tree correcting the errors left by the others.

2

XGBoost combines **gradient-based optimization**, regularization and computational efficiency.

3

Careful **EDA** and **leakage-safe** preprocessing come before any model.

4

The **decision threshold** can change how useful a classifier really is.



Thank You — Questions?

